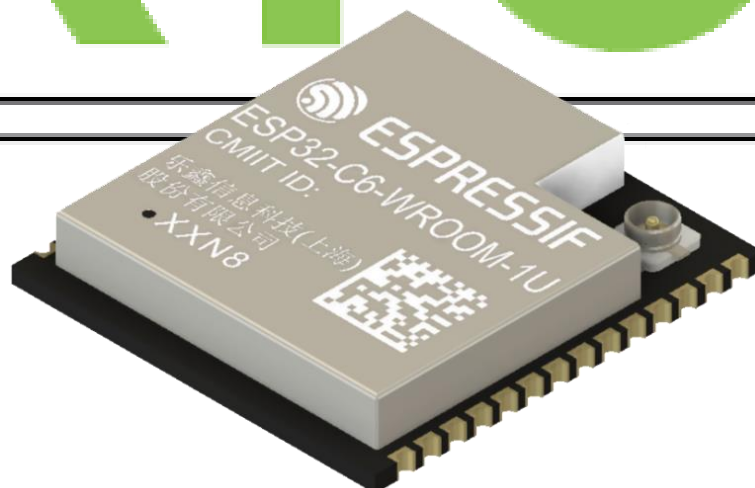


30-Day FreeRTOS Course for ESP32 Using ESP-IDF

(Day 8)

“Using Queues with
ISRs”

freeRTOS



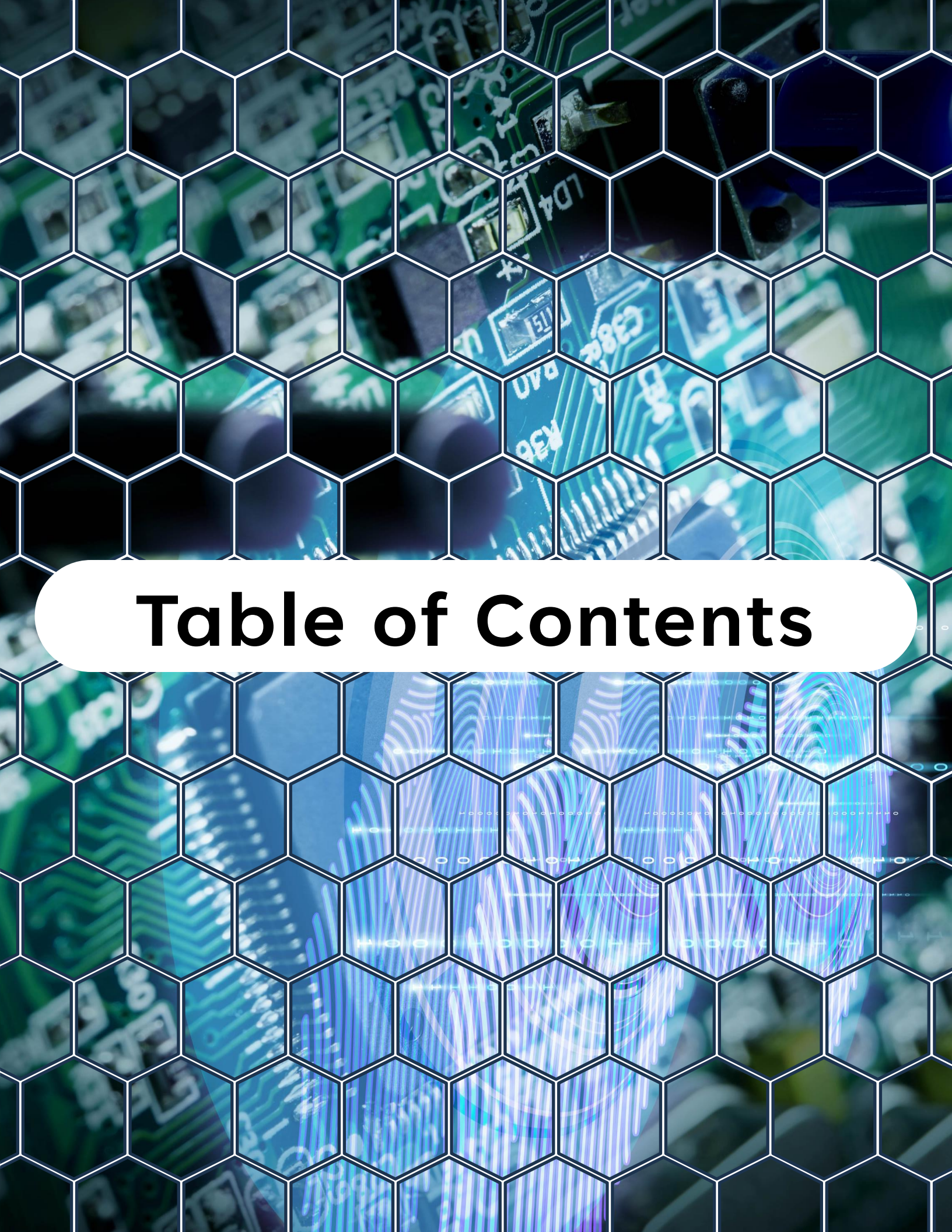
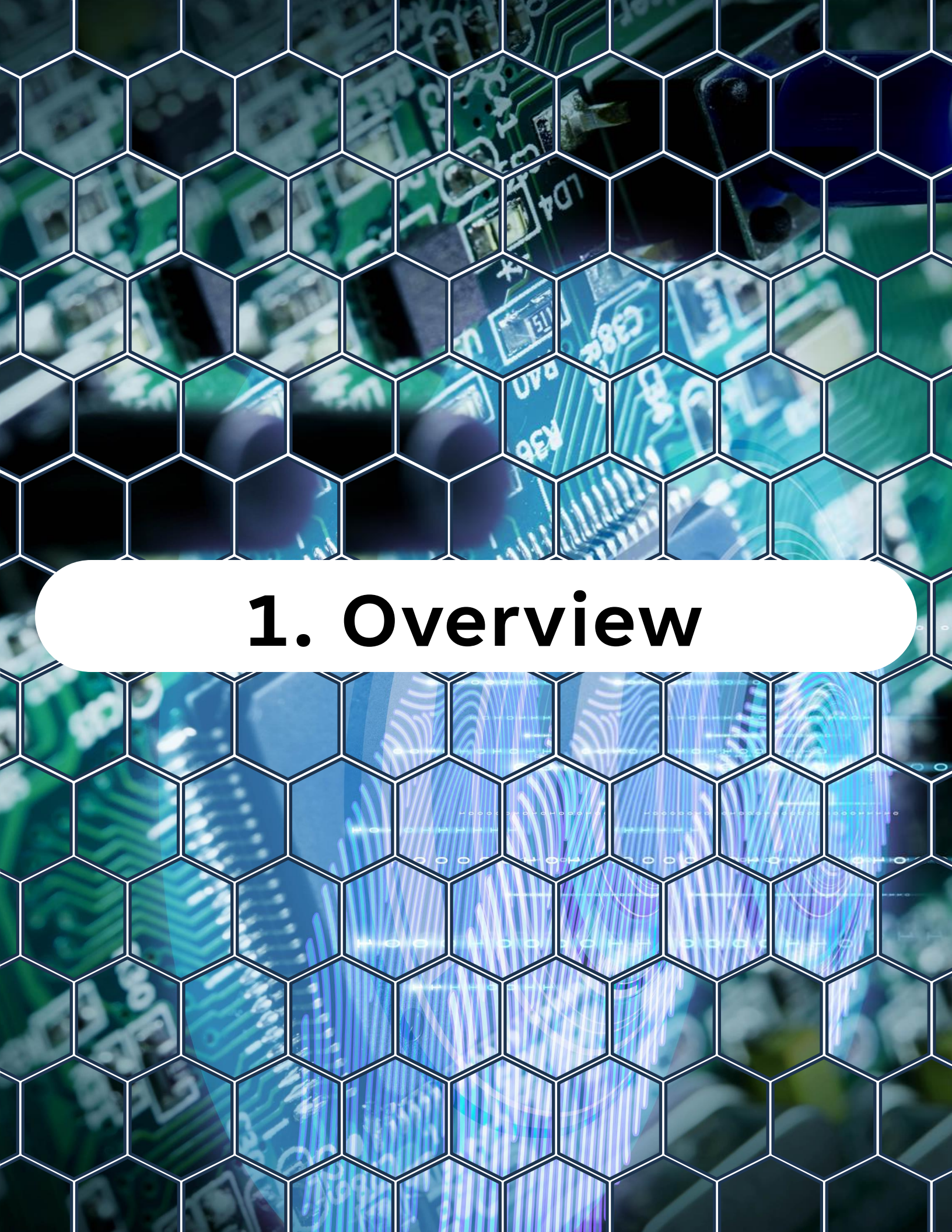


Table of Contents

Table of Contents

1. Overview
2. Why Use Queues with ISRs?
3. ISR-Safe Queue Functions
4. Example – Button ISR Sending to Queue
5. Best Practices
6. Summary
7. Challenge for Today



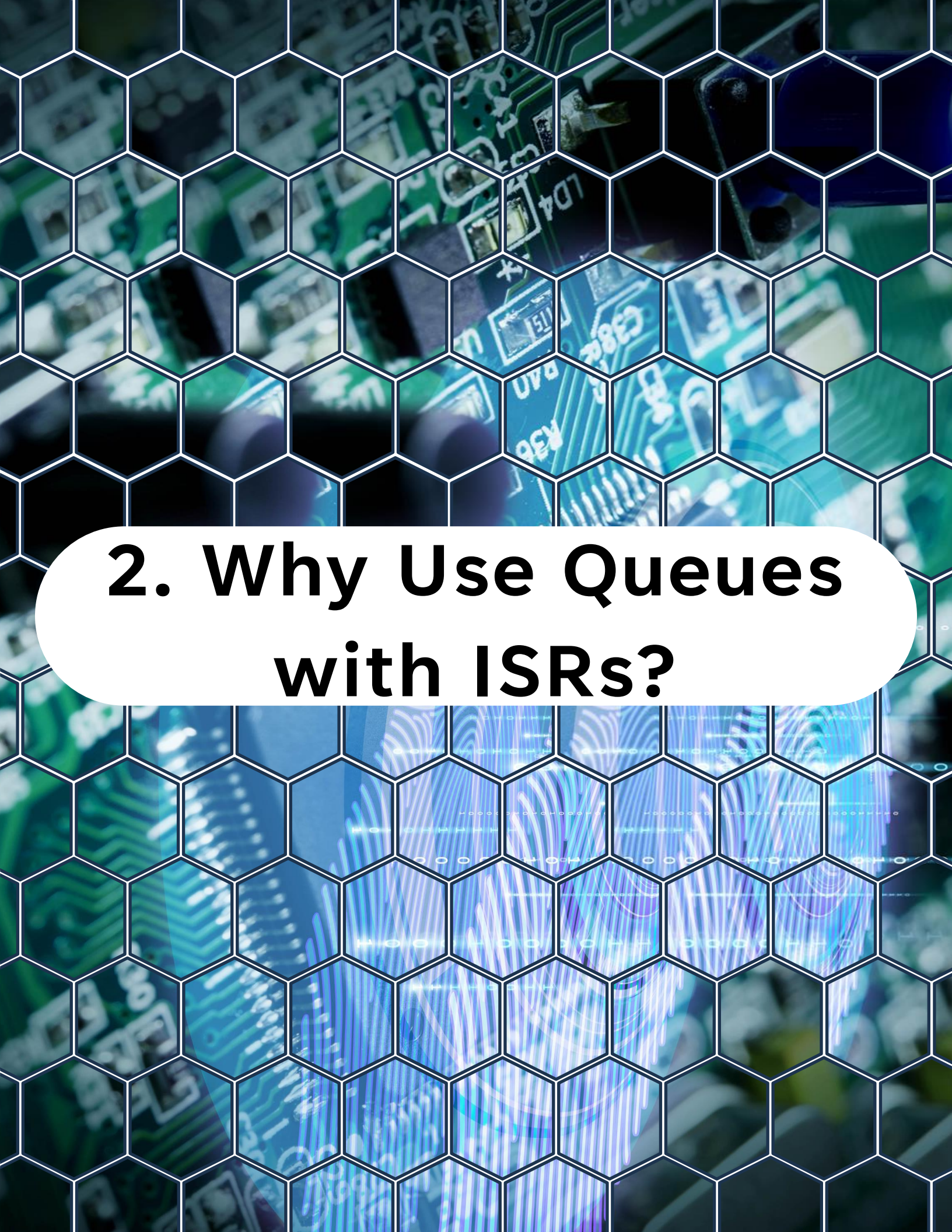
1. Overview

1. Overview

On **Day 9**, you'll learn:

- Why queues are commonly used for **ISR-to-task communication**
- How FreeRTOS provides **ISR-safe APIs** for queues
- How to safely send data from an ISR to a task
- A real example: **Button ISR → Queue → Task**
- Best practices for keeping ISRs short and efficient

By the end of this lesson, you'll know how to connect interrupts to tasks using queues in ESP-IDF with FreeRTOS.



2. Why Use Queues with ISRs?

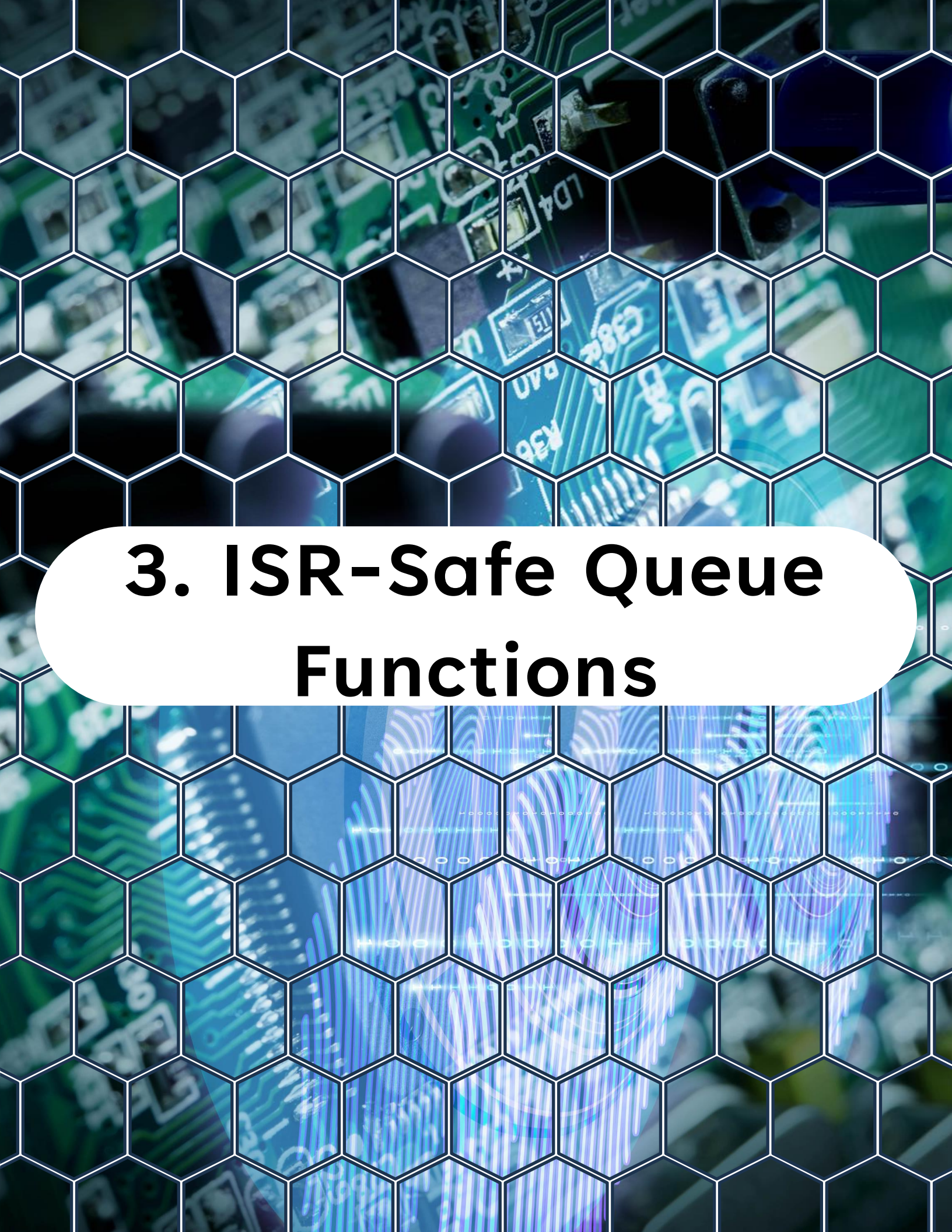
2. Why Use Queues with ISRs?

ISRs (Interrupt Service Routines) should be:

- **Fast** → do minimal work
- **Non-blocking** → never wait for resources
- **Safe** → avoid complex operations (like logging or long computations)

 Instead of doing the work in the ISR, the **ISR** sends a message to a queue, and a task handles the real processing.

This keeps ISRs lean and the system responsive.



3. ISR-Safe Queue Functions

3. ISR-Safe Queue Functions

FreeRTOS provides special versions of queue functions for ISRs:

Send to queue from ISR

```
1 BaseType_t xQueueSendFromISR(QueueHandle_t xQueue,  
2                             const void *pvItemToQueue,  
3                             BaseType_t *pxHigherPriorityTaskWoken);
```

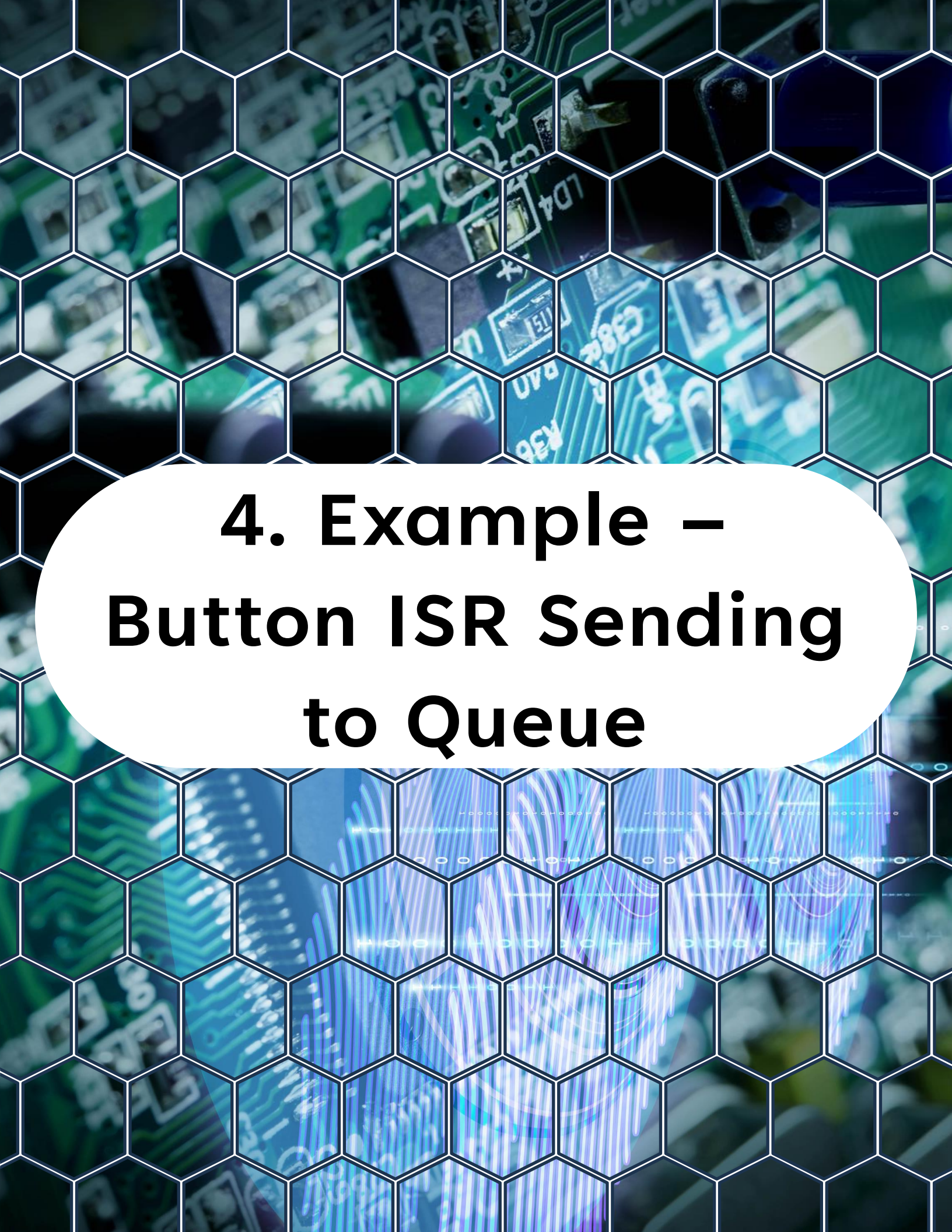
Receive from ISR (rarely used, tasks usually do the receiving)

```
1 BaseType_t xQueueReceiveFromISR(...);
```

Notes:

`pxHigherPriorityTaskWoken`:

- Set to `pdTRUE` if sending to the queue unblocked a higher-priority task.
- If true, call `portYIELD_FROM_ISR()` to force a context switch immediately.

The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and resistors, some of which are labeled with text like 'LD4', 'C382', 'U7A', and 'K36'. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb effect. In the center, a white rounded rectangle contains the title text in a bold, black, sans-serif font.

4. Example – Button ISR Sending to Queue

4. Example – Button ISR Sending to Queue

```
1  #include <stdio.h>
2  #include <inttypes.h>
3  #include "freertos/FreeRTOS.h"
4  #include "freertos/task.h"
5  #include "freertos/queue.h"
6  #include "driver/gpio.h"
7  #include "esp_log.h"
8
9  #define BUTTON_GPIO 0    // Example: BOOT button on many ESP32 boards
10 #define TAG "DAY9"
11
12 QueueHandle_t button_queue;
13
14 // ISR handler
15 static void IRAM_ATTR button_isr_handler(void *arg) {
16     uint32_t gpio_num = (uint32_t) arg;
17     BaseType_t xHigherPriorityTaskWoken = pdFALSE;
18
19     // Send GPIO number to the queue (non-blocking, ISR safe)
20     xQueueSendFromISR(button_queue, &gpio_num, &xHigherPriorityTaskWoken);
21
22     // If a higher-priority task was woken, yield immediately
23     if (xHigherPriorityTaskWoken) {
24         portYIELD_FROM_ISR();
25     }
26 }
27
28 // Task to process button presses
29 void button_task(void *pvParameter) {
30     uint32_t io_num;
31     while (1) {
32         // Wait indefinitely until ISR sends a message
33         if (xQueueReceive(button_queue, &io_num, portMAX_DELAY)) {
34             ESP_LOGI(TAG, "Button pressed on GPIO %" PRIu32 , io_num);
35
36             // Debounce or handle logic here
37
38         }
39     }
40 }
```

4. Example – Button ISR Sending to Queue

```
42 void app_main() {
43     // Create queue for GPIO events
44     button_queue = xQueueCreate(10, sizeof(uint32_t));
45
46     // Configure button pin
47     gpio_config_t io_conf = {
48         .intr_type = GPIO_INTR_NEGEDGE, // Trigger on falling edge
49         .mode = GPIO_MODE_INPUT,
50         .pin_bit_mask = (1ULL << BUTTON_GPIO),
51         .pull_up_en = GPIO_PULLUP_ENABLE,
52         .pull_down_en = GPIO_PULLEDOWN_DISABLE
53     };
54     gpio_config(&io_conf);
55
56     // Install ISR service
57     gpio_install_isr_service(0);
58     gpio_isr_handler_add(BUTTON_GPIO, button_isr_handler, (void *) BUTTON_GPIO);
59
60     // Start button processing task
61     xTaskCreate(button_task, "ButtonTask", 2048, NULL, 10, NULL);
62 }
```

Expected Behavior

When you press the button connected to GPIO0:

- The **ISR triggers**.
- The ISR sends the GPIO number into the queue.
- The **button task** wakes up, receives the event, and logs a message.

You'll see in the serial monitor:

```
I (1234) DAY9: Button pressed on GPIO 0
```




5. Best Practices

5. Best Practices

✓ **Keep ISRs short:** only send signals (like queue messages), not logic.

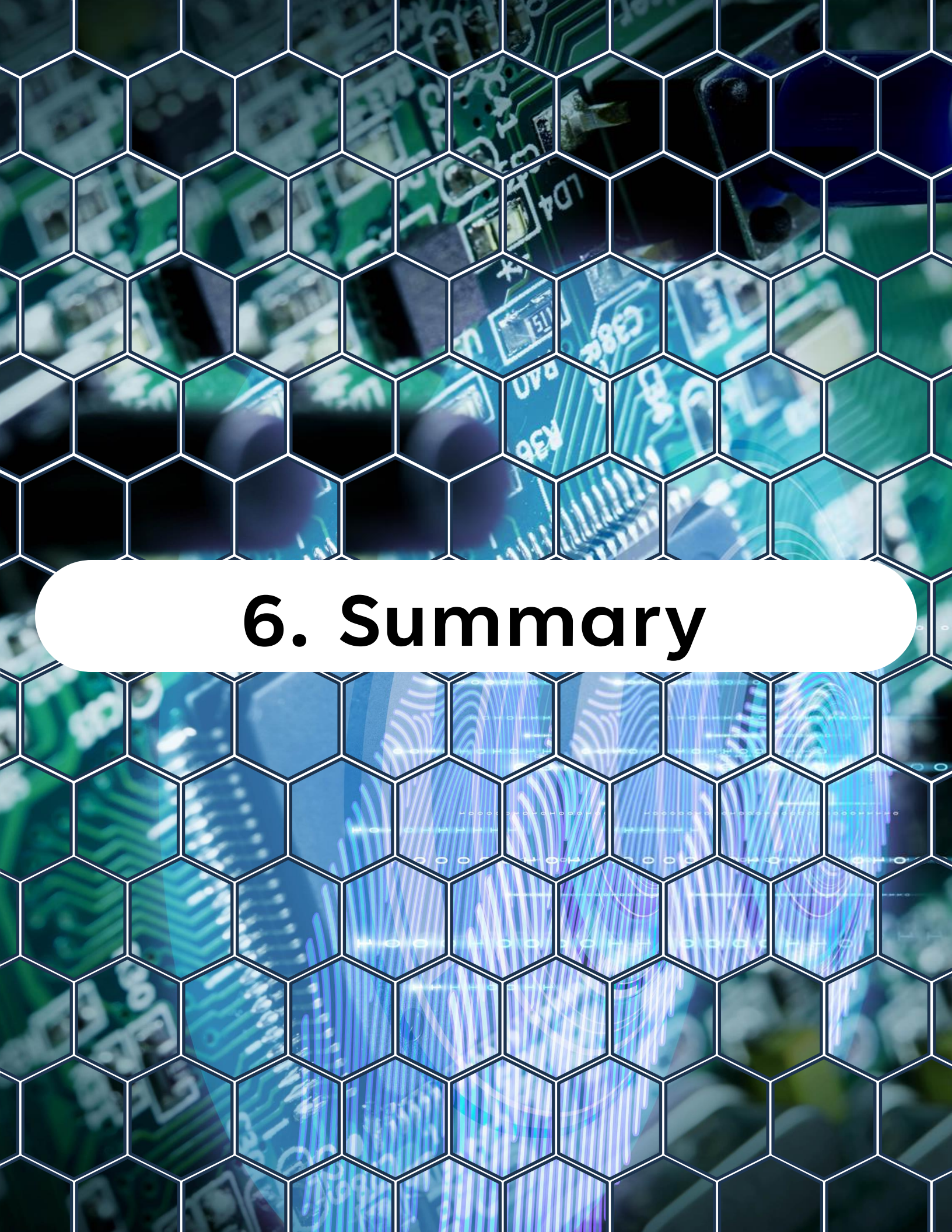
✓ **Use ISR-safe functions**

(xQueueSendFromISR, not xQueueSend).

✓ **Prioritize tasks correctly:** the consumer task should have a high enough priority to handle events quickly.

✓ **Consider debouncing:** use software (in the task) or hardware if dealing with mechanical buttons.

✓ **Use `portYIELD_FROM_ISR()`:** ensures immediate task switch if needed.



6. Summary

6. Summary

- ISRs should be fast and non-blocking.
- Use queues for safe ISR-to-task communication. Always use the **ISR-safe queue functions**.
- Let the task do the heavy lifting, keeping ISRs lightweight.



7. Challenge for Today

7. Challenge for Today

Extend today's project by:

1. Adding a **second button** on a different GPIO.
2. Modify the ISR to send **both GPIO numbers** to the queue.
3. In the task, detect which button was pressed and print a different message.